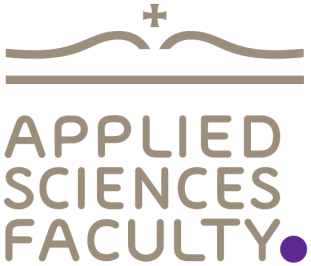
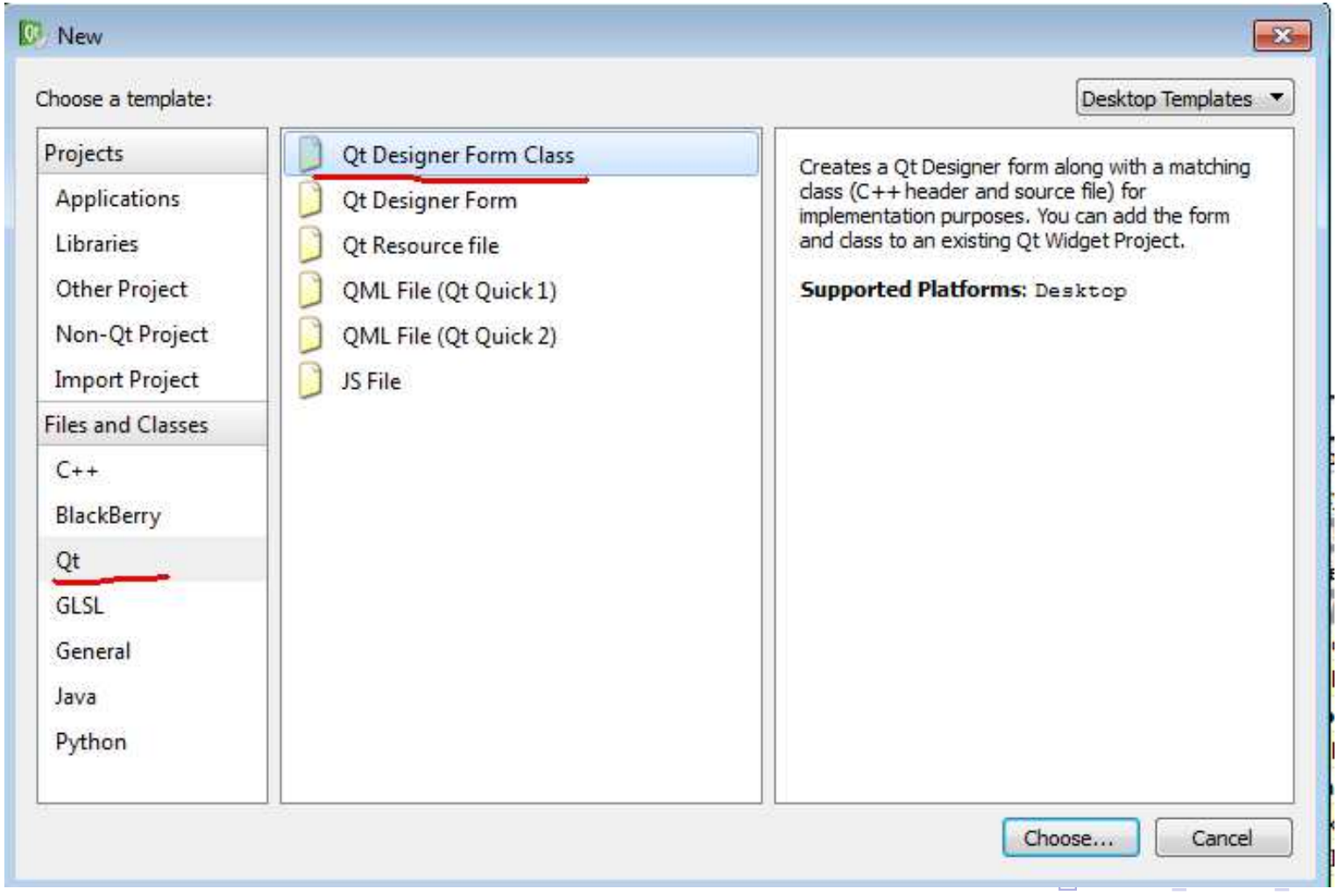
C++ — додаток. Qt. Lecture 6 — користувацькі діалоги на прикладі текстового редактору. Пошук та заміна



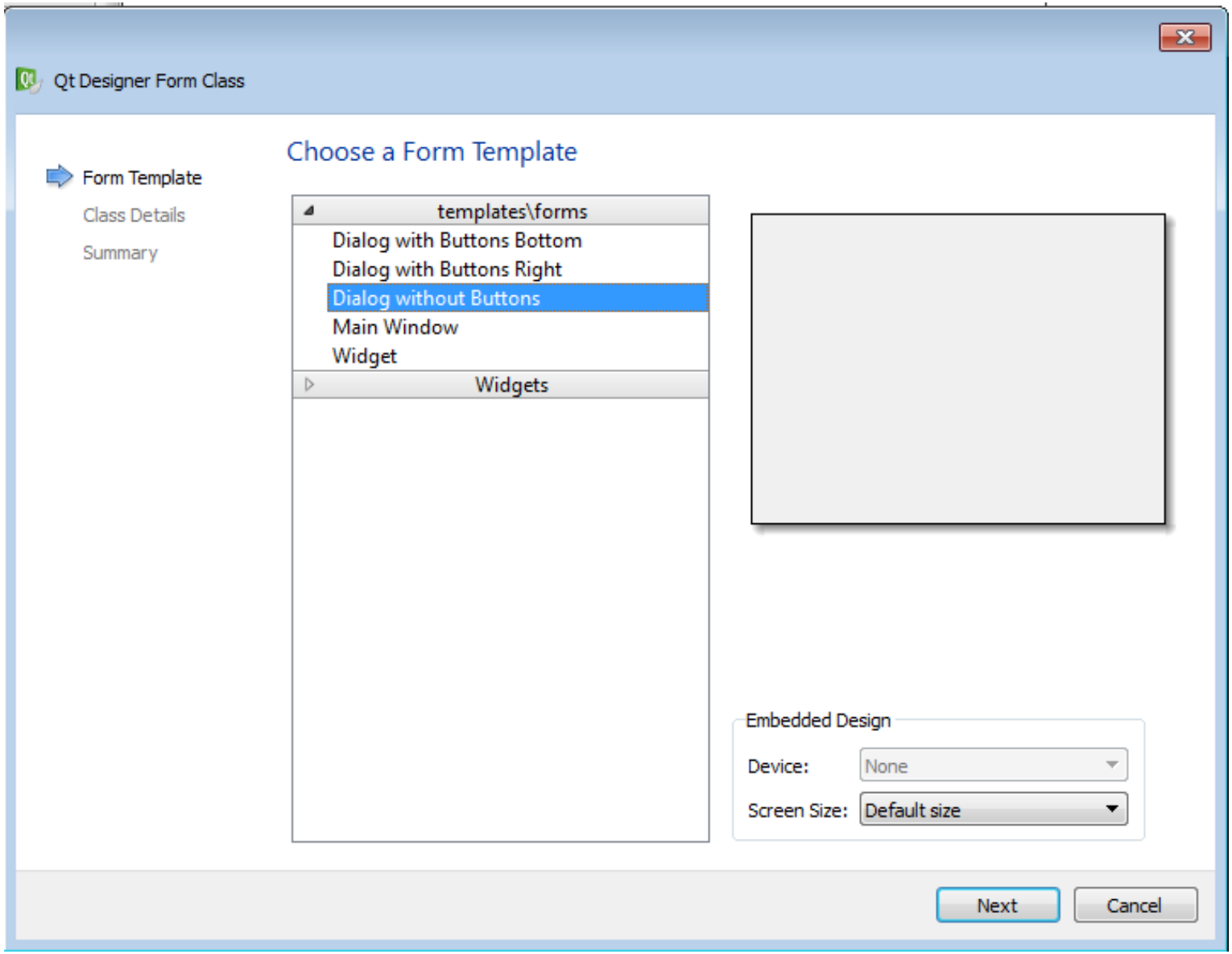


©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

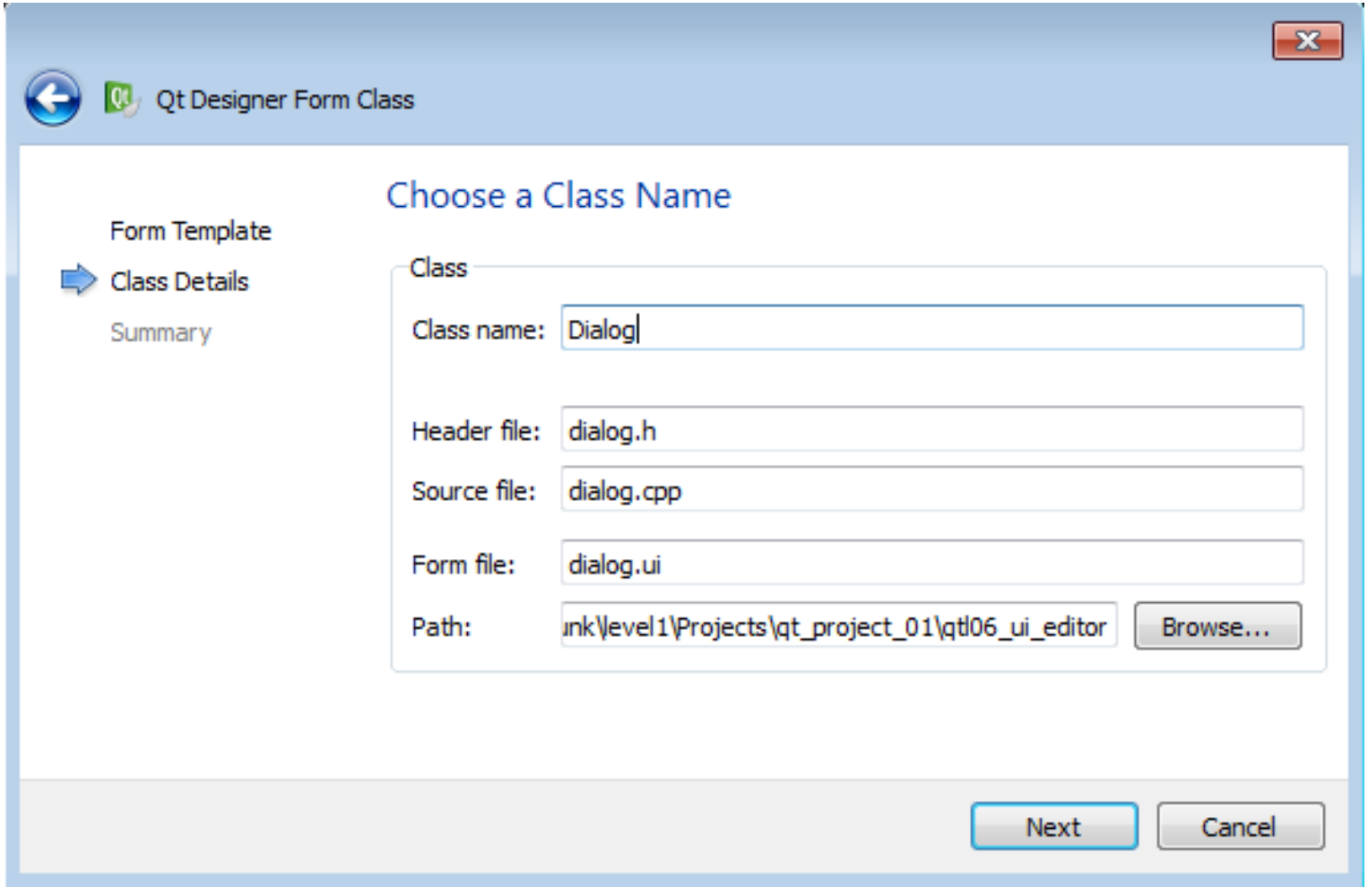
- Діалог пошуку по тексту та діалог заміни нам доведеться самостійно. Почнемо із пошуку.
- Щоб не писати вручну, можна скористатися засобами редактору форм, так само, як ми створювали головне вікно програми.
- File -> New File or Project -> Qt/Qt Designer Form class:



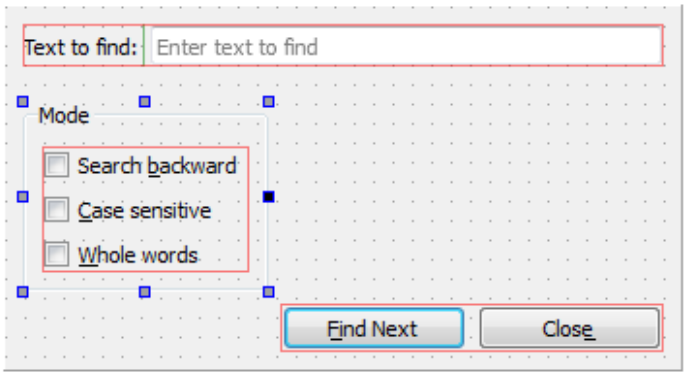
Далі вибираємо вид шаблону форми. Нас цікавить той чи інший діалог

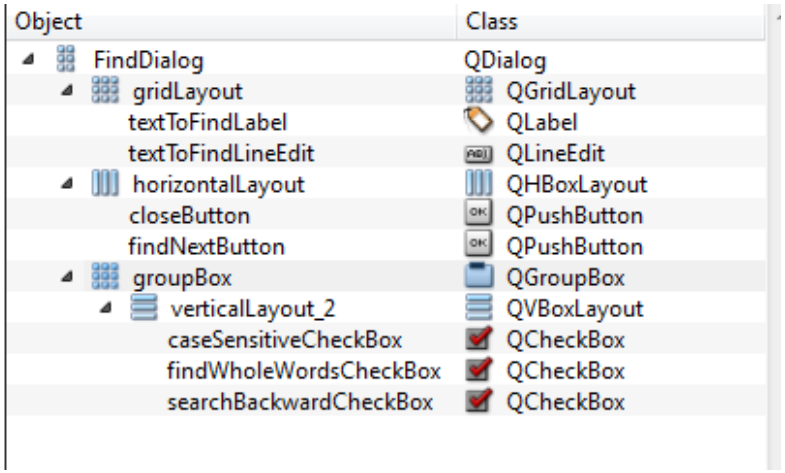


Та налаштовуємо імена її файлів:



- Будувати діалог можна по різному. Нам потрібне поле тексту для пошуку (QLineEdit — про цей клас трішки пізніше), підпис до нього (звичний QLable), кнопки "Знайти" та "Закрити" і, нарешті, ті чи інші засоби вибору режимів пошуку. Скажімо, набір QCheckBox-ів (про які теж пізніше).
- Простір для фантазії тут великий — можна робити по різному. Наприклад, так:





- QCheckBox-и згруповані за допомогою допоміжного класу QGroupBox.

- Створена форма дасть нам кілька файлів — заголовок, реалізацію, ui-файл та отримані з нього, за допомогою uic, .cpp та .h файли. Розглянемо їх.

find_dialog.h:

```
namespace Ui {
class FindDialog;
}

class FindDialog : public QDialog
{
    Q_OBJECT
public:
    explicit FindDialog(QWidget *parent = 0);
    ~FindDialog();

signals:
    void find(const QString &str, QTextDocument::FindFlags ff);

private slots:
    void on_actionFindNext_triggered();

private:
    Ui::FindDialog *ui;
};
```

Все дуже схоже на код класу головного вікна програми.

Реалізація (з врахуванням поки не розглянутих компонент) теж звична:

```
#include "find_dialog.h"
#include "ui_find_dialog.h"

FindDialog::FindDialog(QWidget *parent) :
    QDialog(parent),      ui(new Ui::FindDialog)
{
    ui->setupUi(this);
}

FindDialog::~FindDialog()
{
    delete ui;
}
   ..............
```

Реалізація (з врахуванням поки не розглянутих компонент) теж звична:

```
    ..............
void FindDialog::on_actionFindNext_triggered()
{
    QString textToFind = ui->textToFindLineEdit->text();
    if (!textToFind.isEmpty())
    {
        QTextDocument::FindFlags ff = 0;
        ff = ui->searchBackwardCheckBox->isChecked() ?
                ff | QTextDocument::FindBackward : ff;
        ff = ui->caseSensitiveCheckBox->isChecked() ?
                ff | QTextDocument::FindCaseSensitively : ff;
        ff = ui->findWholeWordsCheckBox->isChecked() ?
                ff | QTextDocument::FindWholeWords : ff;
        emit find(textToFind, ff);
    }
}
```

- Перш ніж зануритися в невідомі поки компоненти та засоби, розглянемо, як можна створювати діалог.
- Підходів можна запропонувати декілька: створювати діалог при запуску програми, показувати його лише за потреби; створювати діалог за потреби, знищувати, коли потреба в ньому зникає; створювати за першого виклику.
- Кожен спосіб має свої сильні та слабкі сторони. Створити діалог наперед — максимально просто, мінус в тому, що для великої програми із багатьма діалогами це повільно і марнотратно. Створювати і знищувати за потреби вимагає акуратної реалізації — щоб не витікала пам'ять та змушує писати багато коду (наприклад, діалог сам не пам'ятатиме свої попередні координати, а користувачу, зазвичай, хотілося б, щоб діалог з'являвся там, де він його залишив минулого разу). Створення за першої вимоги теж не позбавлене недоліків — слід акуратно відслідковувати, коли створювати. Хоча, він, напевне, найзручніший. Крім того, останніх двох підходів стосується проблема вчасного знищення діалогів — щоб, скажімо, сигнал не прийшов вже знищеному діалогу.
- Детальніше про ці підходи, див. відповіді, наприклад, тут: `http://stackoverflow.com/questions/7069282/qt-modeless-dialog-destruction`.

- В даному прикладі вибрано найпримітивніший підхід — діалог створюється при запуску:

```
class  MainWindow : public QMainWindow
{
        ..........................
     QScopedPointer<FindDialog> findDialog;
        ..........................
};

MainWindow::MainWindow(QWidget *parent) :
        ..........................
     findDialog(new FindDialog),
        ..........................
```

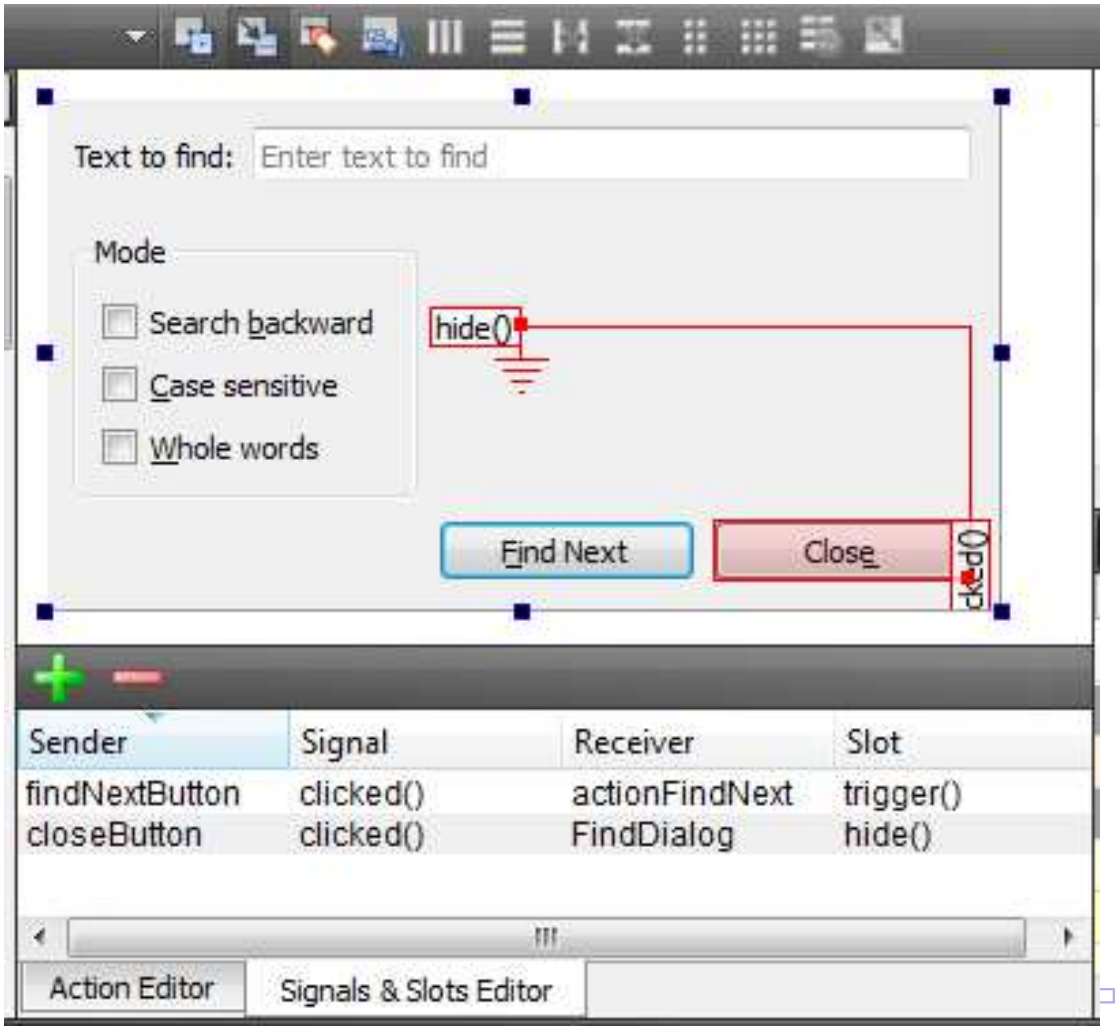
- Для збереження вказівника на діалог використано так званий розумний вказівник, про які детальніше теж трішки пізніше.

- Коли тим чи іншим способом викликано дію (Action) actionFind, її обробник робить наступне:

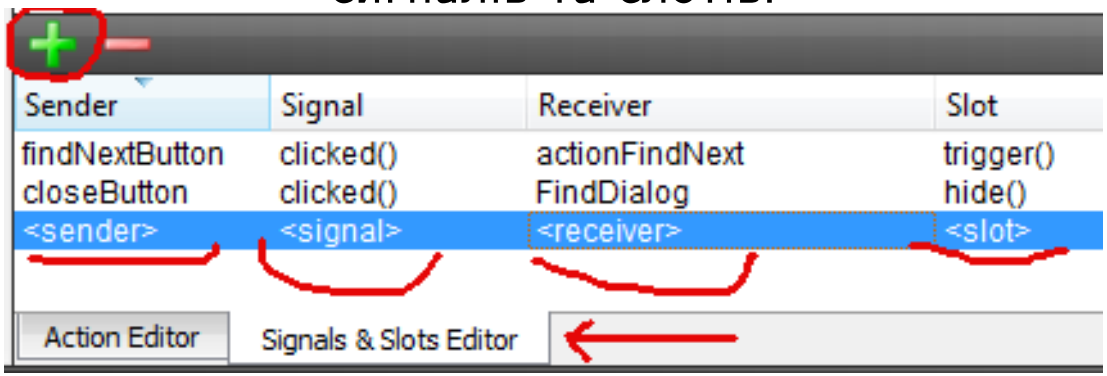
```
void MainWindow::on_actionFind_triggered()
{
    findDialog->show();
    findDialog->activateWindow();
}
```

- Методу show(), успадкованого від QDialog, достатньо, щоб показати і активувати діалог. Однак, він вже може бути видимим, просто не в фокусі, тоді придасться метод activateWindow().
- Далі діалог "крутиться" у власному циклі обробки повідомлень.

- Код його реакції на кнопку "Find Next" ми вже бачили, хоча він потребує детального аналізу. Але коду реакії на "Close" не було.
- Це можливо завдяки візуальним засобам редагування зв'язку слотів та сигналів. Перейти в цей режим можна, натиснувши клавішу F4, поки знаходимося у вікні редактора форм. Повернутися до редагування їх вигляду можна,натиснувши F3:



Того ж результату можна досягнути за допомогою редактору сигналів та слотів:



- За допомогою червоного плюса можна створити новий зв'язок. Тоді в таблиці можна вибрати чий та який саме сигнал має бути пов'язаний із чиїм та яким саме слотом.
- Детальніше тут його не буду описувати — пробуйте, експериментуйте, читайте документацію.

- Пошук в QTextEdit здійснюється за допомогою методу find():

```
bool QTextEdit::find(const QString & exp,
    QTextDocument::FindFlags options = 0);

bool QTextEdit::find(const QRegExp & exp,
    QTextDocument::FindFlags options = 0)
```

- Як видно, він вміє шукати з використанням регулярних виразів. Однак, зараз нас цікавить пошук звичайних під-стрічок.
- Режимом пошуку задається options, яка комбінується побітовим або ("|") з значень перерахування QTextDocument::FindFlags: **FindBackward**, FindCaseSensitively, FindWholeWords.
- Значить, слот пошуку якимось чином повинен отримувати цю інформацію від діалогу.

- Якраз на такі випадки сигнал може передавати параметри, а слот може їх приймати:

```
class FindDialog : public QDialog{
    .....
signals :
    void find(const QString &str, QTextDocument::FindFlags ff);
    .....

class MainWindow : public QMainWindow{
    .....
private slots :
    void onFind(const QString &str, QTextDocument::FindFlags ff);
    .....
```

- Приєднуємо ми такі слоти і сигнали звичним способом:

```
connect( findDialog .data (),
    SIGNAL(find(const QString &, QTextDocument::FindFlags)),
    this ,
    SLOT(onFind(const QString &,QTextDocument::FindFlags)));
```

data() в findDialog.data() стосується розумних вказівників, про це далі.

- Генеруються вони ось наступним чином. Діалог готує інформацію для сигналу, в нашому випадку — стрічку для пошуку та прапорці режиму пошуку, потім генерує сигнал з використанням ще однієї директиви moc — emit:

```
void FindDialog :: on_actionFindNext_triggered(){
        ..............................
    QTextDocument::FindFlags ff = 0;
        ..............................
    emit  find (textToFind,  ff );
        ..............................
}
```

- Таким чином, діалог сам пошуком не займається. Він збирає інформацію про вибір користувача та посилає її сигналом. Пошуком займається відповідний слот.

Детальніше про сигнали та слоти

- Сигнали та слоти можуть бути описані у відповідних частинах опису класів, які є нащадками QObject. C++ код для їх реалізації генерується moc.
- Генеруються сигнали макросом moc, emit. Генерувати сигнал може хто завгодно, але рекомендується, щоб це робив лише клас, де вони оголошені та його нащадки.
- Сигнали можуть приймати параметри, але завжди нічого не повертають! (Тобто, тип значення, яке повертається — void.)
- Так як код для сигналів пишеться moc, **їх реалізації не повинно бути в cpp-файлах проекту**!
- Як тільки сигнал згенеровано, виклик відповідних слотів відбувається негайно, як звичайних функцій — незалежно від циклу обробки подій. Код, що згенерував сигнал, продовжиться після того, як всі слоти завершилися. Це можна змінити, використавши режим приєднання слота Qt::QueuedConnection (див. далі). Тоді код продовжить виконання зразу, а сигнали будуть викликані пізніше.
- Якщо до сигналу приєднано декілька слотів, вони викличуться один за одним, в порядку приєднання.

- Якщо слот і сигнал приєднати декілька раз — за кожної появи сигналу, слот буде викликано стільки раз, скільки їх приєднували. Цьому можна запобігти, використавши режим Qt::UniqueConnection, наприклад:

```
connect( findDialog .data (),
 SIGNAL(find(const QString &, QTextDocument::FindFlags)),
 this , SLOT(onFind(const QString &,QTextDocument::FindFlags)),
        Qt::UniqueConnection );
```

- Слоти — звичайні функції C++, за винятком того, що їх можна приєднати до сигналу. Тобто, викликати їх безпосередньо теж можна.
- Слоти і сигнали можна приєднувати, незалежно від їх прав доступу — тобто, будь-який сигнал може приєднатися як до публічних, так і до приватних чи захищених слотів.
- Слоти можуть бути віртуальними!
- Слід пам'ятати, що в багатопоточній програмі слот, викликаний безпосередньо і слот, викликаний через появу сигналу, до якого він приєднаний, можуть відбуватися в контексті різних потоків!
- Сигнали і слоти можуть містити аргументи по замовчуванню.

Приєднання сигналів до слотів можна здійснити різними способами[1].

- Нехай у нас є сигнал, який викликається, коли екземпляр QObject знищується:

  void destroyed(QObject* = 0);

  Ми можемо приєднати до нього свій слот, якщо за знищення об'єкту слід провести певну чистку. Слот може виглядати так:

  void objectDestroyed(QObject* obj = 0);

- Перший спосіб приєднати — із використанням вказівників на методи:

  ```
  connect(sender, &QObject::destroyed,
                  this, &MyObject::objectDestroyed);
  ```

  При цьому компілятор має можливість перевірити коректність виклику, і, за потреби, здійснити припустимі неявні перетворення аргументів.

[1] Приклади взято з офіційної документації.

- Другий спосіб — використання макросів SIGNAL і SLOT:

```
connect(sender, SIGNAL(destroyed(QObject*)),
                this, SLOT(objectDestroyed(Qbject*)));
```

- Якщо працюється із слотами чи сигналами, які мають аргументи по замовчуванню, то та їх частина, що передається SIGNAL() не повинна бути меншою, ніж передана SLOT(). Тобто, такі варіанти є припустимими:

```
connect(sender, SIGNAL(destroyed(QObject*)),
                this, SLOT(objectDestroyed(Qbject*)));
connect(sender, SIGNAL(destroyed(QObject*)),
                this, SLOT(objectDestroyed()));
connect(sender, SIGNAL(destroyed()),
                this, SLOT(objectDestroyed()));
```

а наступний ні:

```
connect(sender, SIGNAL(destroyed()),
                this, SLOT(objectDestroyed(QObject*)));
```

тому що слот чекатиме на вказівник на QObject, який не буде послано.

- Також дозволено використання лямбда-функції із C++11:

```
connect(sender, &QObject::destroyed,
    [=](){ this->m_objects.remove(sender); });
```

- Взагалі, така форма може використовуватися для приєднання різноманітних функторів:

```
QObject::connect(button, &QPushButton::clicked,
 someFunctor);
```

- Є і багато інших способів. Детальніше див. документацію на різноманітні перевантаження connect: `http://qt-project.org/doc/qt-5/qobject.html#connect`

- Слот може отримати вказівник на об'єкт, який згенерував сигнал, за допомогою функції QObject::sender(). Детальніше див. `http://qt-project.org/doc/qt-5/qobject.html#sender`.

QSignalMapper — 1

- Іноді виникає ситуація, коли слот приєднаний до декількох сигналів, але реагувати на них він повинен по різному. Наприклад[2], в залежності, від якої кнопки прийшов сигнал, вставляти різні літери.
- В таких ситуаціях може бути корисним клас QSignalMapper.
- Всі згадані сигнали відображаються на його екземпляр:

```
signalMapper = new QSignalMapper(this);
signalMapper->setMapping(AButton, QString("A"));
signalMapper->setMapping(BButton, QString("B"));
signalMapper->setMapping(CButton, QString("C"));

connect(AButton, &AButton::clicked,
    signalMapper, &QSignalMapper::map);
connect(BButton, &BButton::clicked,
    signalMapper, &QSignalMapper::map);
connect(CButton, &CButton::clicked,
    signalMapper, &QSignalMapper::map);
```

- Тоді, отримувач приєднується до його сигналу mapped():

```
connect(signalMapper, SIGNAL(mapped(QString)),
        this, SLOT(addText(QString)));
```

QSignalMapper — 2

- QSignalMapper може відображати цілі числа, стрічки чи віджети (QWidget, або й QObject).
- Детальніше див. `http://qt-project.org/doc/qt-5/qsignalmapper.html`

Види під'єднання

- Під'єднання керується останнім параметром connect, типу Qt::ConnectionType:
- Qt::AutoConnection, **використовується по замовчуванню** — якщо отримувач в тому ж потоці, використовує Qt::DirectConnection, інакше — Qt::QueuedConnection.
- Qt::DirectConnection — слот викликається негайно. Виконується він в контексті потоку, який згенерував сигнал.
- Qt::QueuedConnection — слот починає виконуватися після того, як emit повернув керування коду, який його виклика. Слот виконується в контексті потоку-отримувача.
- Qt::BlockingQueuedConnection — те ж, що і попереднє, але emit чекає на завершення отримувача. **Увага!**, якщо отримувач і код, який посилає сигнал, знаходяться в одному потоці, програма просто зависне.
- Qt::UniqueConnection — прапорець, який можна додати (побітовим або), до будь-якого із наведених вище. Він вказує не приєднувати ще раз, якщо даний слот вже приєднано до даного сигналу.

- Для випадків відтермінованого виклику — Qt::QueuedConnection і т.д., Qt повинен знати, як зберігати передані аргументи між викликами.
- Щоб повідомити йому про той чи інший користувацький тип, слід скористатися макросом:

  Q_DECLARE_METATYPE(Type)

  або функцією (якщо реєструється синонім вже відомого системі типу):

  ```
  int qRegisterMetaType(const char * typeName);
  ```

  Наприклад, так:

  ```
  struct MyStruct
  {     ...
  };

  Q_DECLARE_METATYPE(MyStruct)

  typedef QString CustomString;
  qRegisterMetaType<CustomString>("CustomString");
  ```

Детальніше див. документацію:

- `http://qt-project.org/doc/qt-5/signalsandslots.html` — Signals & Slots.
- `http://qt-project.org/doc/qt-5/qsignalmapper.html` — QSignalMapper.
- `http://qt-project.org/doc/qt-5/qt.html#ConnectionType-enum` — enum Qt::ConnectionType.
- `http://qt-project.org/doc/qt-5/qmetatype.html` — QMetaType, зокрема функція qRegisterMetaType() і макрос Q_DECLARE_METATYPE( ).
- `http://qt-project.org/doc/qt-5/threads-qobject.html#signals-and-slots-across-threads` — Signals and Slots Across Threads.
- `http://www.slideshare.net/neeramital/qt-framework-events-signals-threads` – "Qt Framework Events Signals Threads 16,959"
- `http://qt-project.org/doc/qt-5/eventsandfilters.html` — "The Event System".

QLineEdit

- Використовується для введення одного рядка тексту.
- Вміє виконувати стандартні операції редагування — копіювання, вставка, undo, redo, тощо.
- Введений текст можна отримати за допомогою методу text():

```
QString      QLineEdit::text() const;
```

- Встановити текст, повністю замінивши попередній:

```
void      QLineEdit::setText(const QString &);
```

або замінивши лише виділений фрагмент:

```
void QLineEdit:: insert (const QString & newText)
```

- За зміни тексту генерується сигнал:

```
void      QLineEdit::textChanged(const QString & text);
```

Якщо текст змінили чим завгодно, крім setText(), генерується такий сигнал:

```
void QLineEdit::textEdited(const QString & text);
```

- Коли перемістили курсор, генерується:

  void QLineEdit::cursorPositionChanged(**int** old, **int** **new**);

- А коли натиснули Enter:

  void QLineEdit::returnPressed();

- echoMode() дозволяє прочитати, а setEchoMode() — встановити, поточний режим відображення (exa). Режими задаються перерахуванням enum QLineEdit::EchoMode: Normal, NoEcho, Password, PasswordEchoOnEdit. Три останніх використовуються для введення паролів. Відповідно, вони означають відсутність exa, стандартний символ пароля (наприклад *), або символ, поки він вводиться, а потім стандартний символ паролю. Що показується на екрані в такому випадку можна подивитися методом displayText().
- Є знайомий по QTextEdit набір методів, таких як cut(), copy(), paste(), undo(), redo(), selectAll(), clear(), setAlignment().

- Керувати форматом введеного можна за допомогою валідаторів або масок:

```
const QValidator * QLineEdit:: validator () const;
void QLineEdit:: setValidator (const QValidator * v);
QString          QLineEdit::inputMask() const;
 void    QLineEdit::setInputMask(const QString & inputMask);
```

- Маски дозволяють задати, символи якого типу є припустимими. Наприклад, маска для формату введення часу у форматі 4-цифровий рік – 2 цифри місяця – 2 цифри дати, виглядатиме так: `0000-00-00`. Нуль означає, що цифра може бути присутня, але не обов'язкова, дефіс зображає сам себе. Детальніше про коди масок див. тут: `http://qt-project.org/doc/qt-5/qlineedit.html#inputMask-prop`.
- Нащадки QValidator перевіряють коректність введення. Сам він абстрактний, ось його типові нащадки: QIntValidator і QDoubleValidator, які перевіряють, Чи коректне ціле/дробове число було введено. Детальніше див. документацію: `http://qt-project.org/doc/qt-5/qvalidator.html`, `http://qt-project.org/doc/qt-5/qdoublevalidator.html`, `http://qt-project.org/doc/qt-5/qintvalidator.html`, urlhttp://qt-project.org/doc/qt-5/qregexpvalidator.html.

- Традиційно, більше інформації про QLineEdit можна знайти в офіційній документації: `http://qt-project.org/doc/qt-5/qlineedit.html`.

QCheckBox

- QCheckBox це компонента керування (control), яка показує одну галочку, разом із її підписом. Тобто, дозволяє запитати в користувача одну булівську величину. Є нащадком QAbstractButton.
- Подивитися, чи QCheckBox вибрана, можна методом:

  bool QCheckBox::isChecked() **const**;

  Програмно змінити її стан:

  void QCheckBox::setChecked(**bool**);

- Текст підпису встановлюється методом setText():

  void QCheckBox::setText(**const** QString & text)

- QCheckBox може бути трьох-становою, із додатковим станом "ні вибрана, ні не вибрана". Переключити її в такий режим можна за допомогою методу:

  void setTristate (**bool** y = **true**);

  (подивитися — методом isTristate()).

- При зміні стану генерує сигнал:

  void QCheckBox::stateChanged(**int** state);

  Відповідне перерахування enum Qt::CheckState: Unchecked, PartiallyChecked (отой третій, невизначений, стан), Checked.
- Детальніше див. документацію: `http://qt-project.org/doc/qt-5/qcheckbox.html`.

Логічне та фізичне групування

- Для реалізації поведінки, коли із групи QCheckBox може бути вибраною лише одна, можна скористатися абстрактним контейнером QButtonGroup, який не маючи візуального представлення, групує їх логічно. Детальніше див. `http://qt-project.org/doc/qt-5/qbuttongroup.html`.
- QGroupBox надає "коробочку"-фрейм, із заголовком, який групує кілька віджетів візуально. Детальніше див. `http://qt-project.org/doc/qt-5/QGroupBox.html`.
- Обидва класи вище видадуться зовсім знайомими – все ті ж addWidget(), setTitle() і т.д.

Аналіз слоту on_actionFindNext_triggered

- Нарешті загадковий код, наведений раніше:

```
void FindDialog::on_actionFindNext_triggered()
{
    QString textToFind = ui->textToFindLineEdit->text();
    if (!textToFind.isEmpty())
    {
        QTextDocument::FindFlags ff = 0;
        ff = ui->searchBackwardCheckBox->isChecked() ?
                ff | QTextDocument::FindBackward : ff;
        ff = ui->caseSensitiveCheckBox->isChecked() ?
                ff | QTextDocument::FindCaseSensitively : ff;
        ff = ui->findWholeWordsCheckBox->isChecked() ?
                ff | QTextDocument::FindWholeWords : ff;
        emit find(textToFind, ff);
    }
}
```

став зрозумілим: добуваємо стрічку для пошуку, згідно вибраних чек-боксів налаштовуємо режим пошуку, генеруємо сигнал.

Розумні вказівники

- Керування пам'яттю в C++ справа зовсім складна.
- Воно сильно спрощується, якщо всі ресурси, які виділяються, загортати у відповідні класи, деструктори яких звільнятимуть виділені ресурси. Такий підхід називається "Resource Acquisition Is Initialization", або RAII — "Отримання ресурсу є ініціалізація". Див. також `http://uk.wikipedia.org/wiki/Resource_Acquisition_Is_Initialization` та англійську версію цієї ж статті.
- Однак, писати такий клас кожен раз, коли потрібно динамічно виділити щось, незручно. При чому, обійтися без динамічного виділення в ООП програмах майже неможливо.
- Для спрощення роботи у таких випадках було винайдено поняття розумного вказівника — smart pointer.
- Розумний вказівник — сутність, яка поводиться як вказівник (зокрема, для неї перевизначено оператори розіменування та звертання до поля за вказівником на клас чи структуру), але при тому виконує додаткові операції, такі як звільнення пам'яті при виході з області видимості, перевірка виходу за межі, тощо. Див. також `http://uk.wikipedia.org/wiki/Розумні_вказівники` і англійський варіант цієї статті.

- Хоча написати якісний розумний вказівник не так просто, вже є багато готових реалізацій. По перше, це є розумні вказівники з C++11, та їх прототип, який чудово працює із старішими компіляторами з boost, бібліотека `boost::smart_ptr`. По друге, так як згадані реалізації відносно нові, Qt має власні розумні вказівники. Розглянемо, що ж це таке, на їх прикладі.

QPointer

- Шаблон QPointer — простий, "напів-розумний" вказівник, який призначений зберігати вказівники на нащадки QObject. Він поводиться як звичайний вказівник, але коли об'єкт, на який він вказує, знищується (викликається деструктор), вказівник встановлюється в нуль:

```
QPointer<QLabel> label = new QLabel;
label->setText("&Status:");
...
if ( label )
    label->show();
```

  Цитуючи документацію: "Use QPointers like normal pointers and you will not need to read this class documentation."
- QPointer можна вільно копіювати та змішувати із звичайними вказівниками.
- Для перевірки на нульовий вказівник, крім операторів == і !=, існує метод isNull().

QScopedPointer — 1

- QScopedPointer зберігає вказівник на динамічно виділений об'єкт, та звільняє його в своєму деструкторі.
- Використовується якось так:

```
QScopedPointer<QWidget> pw( new QWidget() );
```

- Копіювати чи присвоювати QScopedPointer не можна — відповідні оператори та конструктори заборонені.
- Може виражати константність. Приклад взято з офіційної документації, кожних два рядки еквівалентні.

```
const QWidget *const p = new QWidget();
const QScopedPointer<const QWidget> p(new QWidget());

QWidget *const p = new QWidget();
const QScopedPointer<QWidget> p(new QWidget());

const QWidget *p = new QWidget();
QScopedPointer<const QWidget> p(new QWidget());
```

QScopedPointer — 2

- Видаляти різні вказівники треба по різному — одиничні динамічні змінні за допомогою delete, масиви — delete[], пам'ять, виділену функціями сімейства malloc() — free(), тощо.
- Щоб вказати QScopedPointer, як правильно видаляти, може використовуватися другий параметр шаблону (який, по замовчуванню, вказує видаляти за допомогою delete). Наприклад, видаляти за допомогою delete[]:

```
QScopedPointer<int, QScopedPointerArrayDeleter<int> >
                                        ap(new int[42]);
```

Передбачено наступні варіанти: QScopedPointerDeleter, QScopedPointerArrayDeleter, QScopedPointerPodDeleter — видаляти за допомогою free(), QScopedPointerDeleteLater — для об'єктів, з якими активно працюється в циклі обробки повідомлень, "видалити пізніше", посилає сигнал deleteLater() — див. `http://qt-project.org/doc/qt-5/qobject.html#deleteLater`.
Можна передавати власні класи, які повинні мати статичний метод:

```
void cleanup(T *pointer);
```

QScopedPointer — 3

- QScopedArrayPointer — варіант QScopedPointer , який, по замовчуванню, стирає, використовуючи QScopedPointerArrayDeleter.

QScopedPointer і std::unique_ptr

- Аналогом до QScopedPointer є std::unique_ptr з C++11.
- Основна відмінність в тому, що std::unique_ptr підтримує move-семантику — його можна переміщати із rvalue reference (якщо зовсім примітивно — з тимчасових значень, завдяки чому його можна, наприклад, повернути із функції, хоч він і не має конструктора копіювання)[3]:

```
std::unique_ptr<int> p1(new int(5));
std::unique_ptr<int> p2 = p1; //Помилка компіляції
std::unique_ptr<int> p3 = std::move(p1); //Передає володіння.
// Тепер виділеною памяттю' командує"" p3, а p1 ——— порожній.
p3.reset(); //Звільняє память'
p1.reset(); //Не робить нічого
```

- **Застарілий** std::auto_ptr, на відміну від розглянутих розумних вказівників, має оператор копіювання, який передає володіння — копіює вказівник у приймач, а потім перетворює джерело на нульовий вказівник. Такий собі ерзац-move... НЕ КОРИСТУЙТЕСЯ НИМ! Практика показала — від нього більше мінусів, ніж плюсів.

[3] Приклад взято з вікіпедії

QSharedPointer — 1

- QSharedPointer зберігає "сильний" вказівник спільного володіння (strong shared pointer). Що це значить, зараз стане зрозуміло.
- Як і QScopedPointer, він поводиться, як звичайний вказівник. Крім того, його можна копіювати. Всі копії вказують на ті самі дані.
- Його корисність в тому, що видалення відбувається після того, як знищується **останній об'єкт**, який вказує на ці дані.
- Даний клас є потоко-безпечним. Робота з внутрішнім вказівником та лічильником відбувається атомарно, тому різні потоки можуть звертатися до нього без блокувань. Звичайно, це НЕ стосується роботи із тим, на що даний розумний вказівник вказує!
- Як і QScopedPointer, нормально взаємодіє із константністю.

QSharedPointer — 2

- Метод clear() призводить до того, що даний об'єкт перестає вказувати на "свій" об'єкт. Якщо цей вказівник був останнім — він видаляється.
- Статична функція:

  QSharedPointer<T> QSharedPointer::create()
  QSharedPointer<T> QSharedPointer::create(...)

  створює об'єкт і зразу — розумний вказівник який вказує на нього, одним виділенням пам'яті. Варіант із аргументами існує тільки для C++11 (через необхідність prefect forwarding).
- Метод, який повертає "сирий" вказівник:

  T * QSharedPointer::data() **const**;

  Обережно із ним! Він, по суті, відміняє ідею shared pointer. Його не можна ні видаляти, ні передавати іншим розумним вказівникам!
- isNull() працює так само, як і аналогічна з QScopedPointer — перевіряє, чи вказівник нульовий.

QSharedPointer — 3

- Створюючи цей розумний вказівник, можна передати функцію чи функтор, який відповідатиме за видалення:

  QSharedPointer::QSharedPointer(T * ptr, Deleter deleter);

  Наприклад (взято з офіційної документації):

```
static void doDeleteLater(MyObject *obj)
{
    obj->deleteLater();
}

void otherFunction()
{
    QSharedPointer<MyObject> obj =
        QSharedPointer<MyObject>(new MyObject, doDeleteLater);
    .....
    obj.clear();      // Викликається obj->deleteLater();
}
```

QSharedPointer — 4

- Змусити вказівник вказувати на інший об'єкт:

  void QSharedPointer:: reset (T * t);
  void QSharedPointer:: reset (T * t , Deleter deleter);

- Її варіант без аргументі еквівалентний clear(), існує для сумісності з std::shared_ptr.

  void QSharedPointer:: reset ();

- Для отримання відповідного *слабкого* вказівника (див. далі), можна використати:

  QWeakPointer<T> QSharedPointer::toWeakRef() **const**;

- Є також ціле сімейство перетворень типів, для руху по ієрархії класів: qSharedPointerCast, qSharedPointerDynamicCast, і т.д.

QSharedPointer і std::shared_ptr

- Ідейно ці класи дуже схожі, хоча деталі реалізації відрізняються. Зокрема, std::shared_ptr менш "насичений" допоміжними методами, а деякі еквівалентні методи називаються по іншому.
- Детальніше про див. `http://en.cppreference.com/w/cpp/memory/shared_ptr` і `http://www.cplusplus.com/reference/memory/shared_ptr/`

QWeakPointer — 1

- Кожна копія QSharedPointer продовжує час існування того, на що вона посилається на час свого існування.
- Це не завжди зручно. Наприклад, якщо в грі "Астероїди", кожен астероїд має вказівники на своїх сусідів, наявність таких вказівників не повинна заважати знищувати конкретні астероїди. Якщо всі такі вказівники зробити QSharedPointer, отримаємо циклічну залежність — вказівники на конкретний астероїд завжди існуватимуть (в його сусідах, як мінімум), деструктор викликати неможливо. Скористатися звичайними вказівниками теж не вдасться
- Якраз на такий випадок існують "слабкі" вказівники. В Qt це QWeakPointer.
- QWeakPointer зберігає посилання на QSharedPointer, однак не впливає на час його життя.
- Безпосередньо розіменовувати його не можна — він може вказувати на об'єкт, якого не існує.
- Однак, його можна перетворити на QSharedPointer. Якщо отриманий розумний вказівник не буде нульовим, значить об'єкт ще існує, а тепер продовжить існувати, поки створений QSharedPointer не буде знищено.

QWeakPointer — 2

- Для перетворення використовується метод toStrongRef():

  QSharedPointer<T> QWeakPointer::toStrongRef() **const**;

- Приклад використання слабкого вказівника (взято із офіційної документації):

```
QWeakPointer<int> weakref;
        .....
QSharedPointer<int> strong = weakref.toStrongRef();
if (strong)
    qDebug() << "The value is:" << *strong;
else
    qDebug() << "The value has already been deleted";
```

- Для нього, як і для інших розумних вказівників, існує цілий зоопарк методів, таких clear(), data(), isNull() — по самій природі слабкого вказівника, може стати нульовим в будь-який момент, сімейство методів виду qSharedPointerCast, а також оператори присвоєння, порівняння та перетворення до булівського значення.

QWeakPointer і std::weak_ptr

- Ідейно ці вказівники однакові, хоча конкретний інтерфейс відрізняється. Приклад використання weak_ptr, взятий з Вікіпедії:

```
std::shared_ptr<int> p1(new int(5));
std::weak_ptr<int> wp1 = p1;
{
  std::shared_ptr<int> p2 = wp1.lock();
  // Памятю' тепер володіють і p1 і p2.
  // Аналог QWeakPointer::toStrongRef()
  if(p2) // Перевірка, чи все ще існує те, на що вказуємо
  {
        Існує, робимо щось
  }
} //p2 знищено. Памяттю' знову вололіє лише p1.

p1.reset(); //Видаляємо те, на що посилалися.
 std::shared_ptr<int> p3 = wp1.lock();
 //Отримали порожній shared_ptr.
if(p3){
  Сюди точно не потрапимо
}
```

Розумні вказівники — посилання — 1

- http://en.wikipedia.org/wiki/Smart_pointer — огляд розумних вказівників, див. також український варіант статті.
- http://ootips.org/yonat/4dev/smart-pointers.html і http://dlugosz.com/Repertoire/refman/Classics/Smart%20Pointers%20Overview.html — більш детальній, але доступній, огляди питання.
- http://www.josuttis.com/libbook/cont/countptr.hpp.html — приклад реалізації розумних вказівників.
- http://www.informit.com/articles/article.aspx?p=25264 — детальніше про їх реалізацію, за мотивами славнозвісної книги Александреску "Modern C++ Design".
- http://www.boost.org/doc/libs/1_55_0/libs/smart_ptr/smart_ptr.htm — Розумні вказівника boost.
- http://qt-project.org/doc/qt-5/qpointer.html, http://qt-project.org/doc/qt-5/qscopedpointer.html, http://qt-project.org/doc/qt-5/qscopedarraypointer.html, http://qt-project.org/doc/qt-5/qsharedpointer.html, http://qt-project.org/doc/qt-5/qweakpointer.html, http://qt-project.org/doc/qt-5/qshareddatapointer.html — розумні вказівники Qt.

Розумні вказівники — посилання — 2

- `http://www.cplusplus.com/reference/memory/` — розумні вказівники C++11-раз.
- `http://en.cppreference.com/w/cpp/memory` — розумні вказівники C++11-два.

- Тепер наведений вище код повинен бути значно зрозумілішим:

```
class  MainWindow : public QMainWindow
{
        ..........................
     QScopedPointer<FindDialog> findDialog;
        ..........................
};

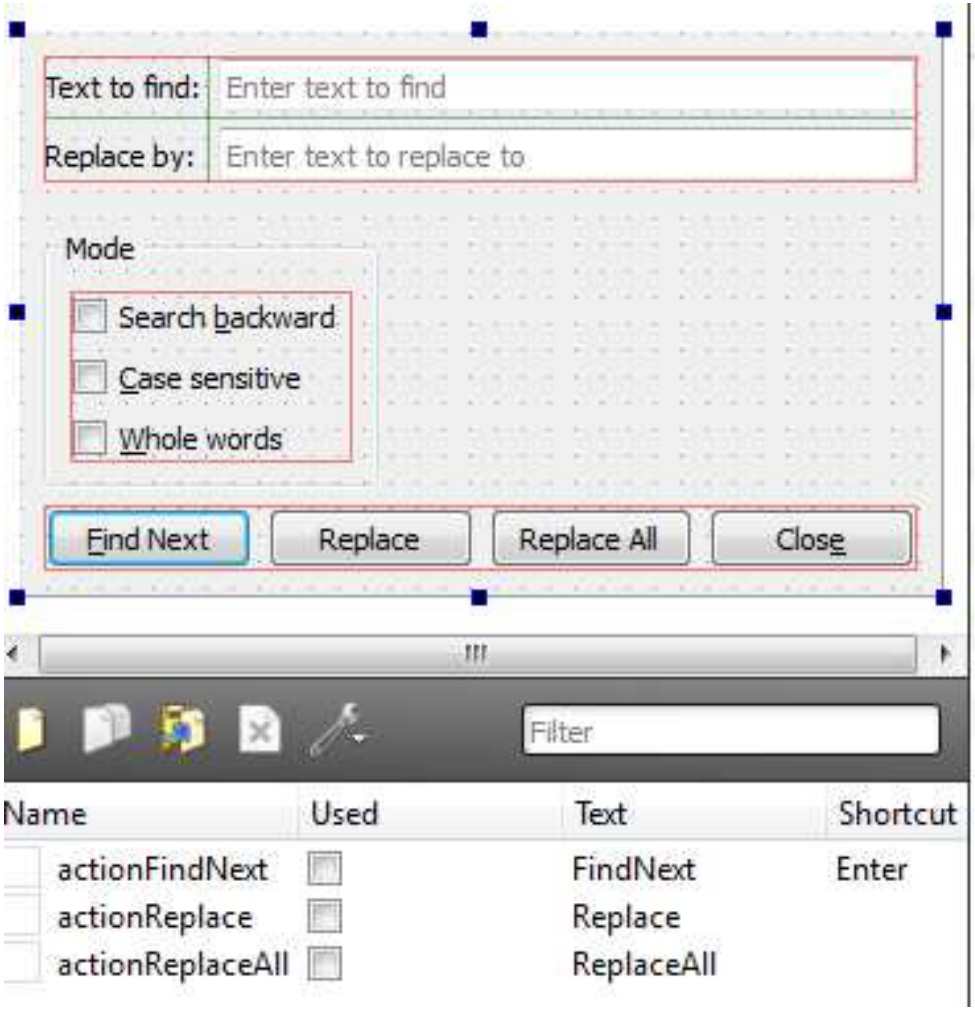
MainWindow::MainWindow(QWidget *parent) :
        ..........................
     findDialog(new FindDialog),
        ..........................
```

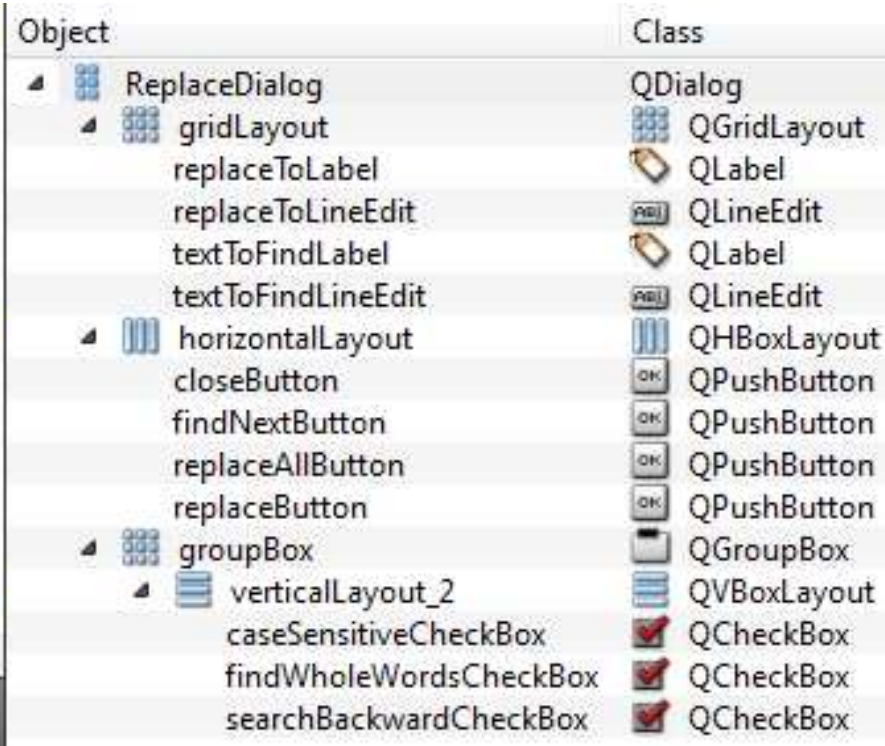
Аналогічно створюємо і діалог заміни:

- **class** MainWindow : **public** QMainWindow
  {
  .....
  QScopedPointer<ReplaceDialog> replaceDialog;
  .....
  };

  MainWindow::MainWindow(QWidget *parent) :
  .....
  replaceDialog(**new** ReplaceDialog)
  .....

Однак, сам діалог дещо складніший





```
class ReplaceDialog : public QDialog
{
    Q_OBJECT

public:
    explicit ReplaceDialog(QWidget *parent = 0);
    ~ReplaceDialog();
signals:
    void find(const QString &str, QTextDocument::FindFlags ff);
    void replace(const QString &strFind, const QString &strReplace,
                 QTextDocument::FindFlags ff);
    void replaceAll(const QString &strFind, const QString &strReplace,
                 QTextDocument::FindFlags ff);

private slots:
    void on_actionFindNext_triggered();
    void on_actionReplace_triggered();
    void on_actionReplaceAll_triggered();

private:
    Ui::ReplaceDialog *ui;
};
```

```
ReplaceDialog::ReplaceDialog(QWidget *parent) :
    QDialog(parent),
    ui(new Ui::ReplaceDialog)
{
    ui->setupUi(this);
}

ReplaceDialog::~ReplaceDialog()
{
    delete ui;
}
```

```
void ReplaceDialog::on_actionFindNext_triggered()
{
    QString textToFind = ui->textToFindLineEdit->text();
    if (!textToFind.isEmpty())
    {
        QTextDocument::FindFlags ff = 0;
        ff = ui->searchBackwardCheckBox->isChecked() ?
             ff | QTextDocument::FindBackward : ff;
        ff = ui->caseSensitiveCheckBox->isChecked() ?
             ff | QTextDocument::FindCaseSensitively : ff;
        ff = ui->findWholeWordsCheckBox->isChecked() ?
             ff | QTextDocument::FindWholeWords : ff;
        emit find(textToFind, ff);
    }

}
```

Як бачимо, сама реалізація нічого видатного не має.

```
void ReplaceDialog::on_actionReplaceAll_triggered()
{
    if (!ui->textToFindLineEdit->text().isEmpty())
    {
        QTextDocument::FindFlags ff = 0;
        // ! Тільки вперед шукаємо ---- від початку до кінця
        ff = ui->caseSensitiveCheckBox->isChecked() ?
             ff | QTextDocument::FindCaseSensitively : ff;
        ff = ui->findWholeWordsCheckBox->isChecked() ?
             ff | QTextDocument::FindWholeWords : ff;

        emit replaceAll(ui->textToFindLineEdit->text(),
                        ui->replaceToLineEdit->text(), ff);
    }
}
```

```
void ReplaceDialog::on_actionFindNext_triggered()
{
    QString textToFind = ui->textToFindLineEdit->text();
    if (!textToFind.isEmpty())
    {
        QTextDocument::FindFlags ff = 0;
        ff = ui->searchBackwardCheckBox->isChecked() ?
             ff | QTextDocument::FindBackward : ff;
        ff = ui->caseSensitiveCheckBox->isChecked() ?
             ff | QTextDocument::FindCaseSensitively : ff;
        ff = ui->findWholeWordsCheckBox->isChecked() ?
             ff | QTextDocument::FindWholeWords : ff;
        emit find(textToFind, ff);
    }

}
```

Відповідна реалізація в головному вікні — 1

Приєднуємо сигнали:

```
connect( replaceDialog.data(),
    SIGNAL(find(const QString &, QTextDocument::FindFlags)),
    this,
    SLOT(onFindForReplace(const QString &,QTextDocument::FindFlags)) );

connect( replaceDialog.data(),
    SIGNAL(replace(const QString &, const QString &,
        QTextDocument::FindFlags)),
    this,
    SLOT(onReplace(const QString &,const QString &,
        QTextDocument::FindFlags)) );

connect( replaceDialog.data(),
    SIGNAL(replaceAll(const QString &, const QString &,
        QTextDocument::FindFlags)),
    this,
    SLOT(onReplaceAll(const QString &,const QString &,
        QTextDocument::FindFlags)) );
```

Сигнали для заміни відрізняються наявністю ще одного аргументу — стрічки, на яку замінити.

- Показуємо діалог пошуку та заміни:

```
void MainWindow::on_actionReplace_triggered()
{
    replaceDialog->show();
    replaceDialog->activateWindow();
}
```

- Пошук — знайти, а що ж замінювати.

```
void MainWindow::onFindForReplace(const QString &str,
                                  QTextDocument::FindFlags ff)
{
    if( ! ui->textEdit->find(str,ff) )
    {
        QMessageBox::warning(replaceDialog.data(), tr("Warning"),
                             QString(tr("String \"%1\" not found.").arg( str )));
    }
}
```

- Заміна трішки хитріша — якщо щось виділено, можна замінити. За виділення відповідає курсор. Тому, перевіряємо, запитавши поточний курсор – а чи він hasSelection(). Інакше — шукаємо, а чи є що замінювати.

```
void MainWindow::onReplace(const QString &strFind,
        const QString &strReplace, QTextDocument::FindFlags ff)
{
    if( ! ui->textEdit->textCursor().hasSelection() )
    {
        emit onFindForReplace(strFind, ff );
    }else
    {
        ui->textEdit->textCursor().insertText( strReplace );
        emit onFindForReplace(strFind, ff );
    }
}
```

Якщо так — заміняємо його ж, курсора, методом, insertText().

Замінити всі — 1

- Зберігаємо поточне положення курсору та переходимо на початок (шукатимемо вниз):

```
void MainWindow::onReplaceAll(const QString &strFind,
      const QString &strReplace, QTextDocument::FindFlags ff)
{
    int currentPos = ui->textEdit->textCursor().position();
    ui->textEdit->moveCursor(QTextCursor::Start);
```

Замінити всі — 2

- Далі, в циклі, замінюємо, підраховуючи, скільки ж замінили:

```
int replacedN=0;
int replaced_before_cursor=0;
while ( ui->textEdit->find(strFind,ff) )
{
    ui->textEdit->textCursor().insertText( strReplace );
    ++replacedN;
    if ( ui->textEdit->textCursor().position() <=
            currentPos + (replaced_before_cursor+1) *
             ( strReplace.size() - strFind.size() )
        )
         ++replaced_before_cursor;
}
```

Оцей складний код, який стосується `replaced_before_cursor` всього лиш виконує підрахунки, щоб після заміни розташувати курсор хоча б приблизно там, де він був до початку замін.

Замінити всі — 3

- Повідомляємо інформацію, скільки ж ми назамінювали:

```
QMessageBox::information(replaceDialog.data(), tr("Information"), \
        QString(tr("String \"%1\" replaced %2 times.").arg(strFind )
```

- На завершення — повертаємо курсор до тієї частини тексту, де він знаходився до початку заміни:

```
// textCursor() повертає копію курсора!
QTextCursor cc = ui->textEdit->textCursor();
// Коректуємо на зміну довжини тексту
currentPos += replaced_before_cursor*
    ( strReplace.size() - strFind.size() );
cc.setPosition( currentPos );
// Робимо змінений курсор поточним
ui->textEdit->setTextCursor( cc );
```

- З нашими діалогами пошуку та заміни — все.
- Про ще одну, просту та достатньо зручну реалізацію діалогів пошуку та заміни див. тут: `https://gitorious.org/qtfindreplacedialog` — "QtFindReplace dialog is an implementation of a Find/Replace dialog reusable in text edit based qt applications."
- Детальніше про діалоги див. тут: `http://qt-project.org/doc/qt-5/QDialog.html` та в його нащадках.
- Цінний приклад розробки схожих програм: "Address Book Tutorial", `http://qt-project.org/doc/qt-4.8/tutorials-addressbook.html`, зокрема, глава "Part 5 - Adding a Find Function": `http://qt-project.org/doc/qt-4.8/tutorials-addressbook-part5.html`.
- Ще один приклад — Application Example: `http://qt-project.org/doc/qt-4.8/mainwindows-application.html`.

Домашнє завдання

Викласти все в лекціях неможливо. Володіння Qt вимагає активної практики. Сам редактор можна розширяти у багатьох напрямках. Спробуйте!

- Діалоги ми розглядали лише побіжно. Для кращого ознайомлення поекспериментуйте із ними.
- Додайте до редактора засоби виводу статистики тексту — кількості слів, рядків, літер, пробільних символів (whitespaces), тощо. Зробити це можна, як за допомогою пункту меню, що виводить діалог, так і за допомогою користувацьких компонент, які постійно видимі.
- Ще одне просте покращення — до діалогу заміни додайте можливість пошуку не тільки від початку, але і від поточного положення курсору – вверх чи вниз, в залежності від відповідного прапорця. Додайте можливість перепитатися в користувача — чи продовжувати далі, якщо дійшли до початку-кінця документу.
- Пошук та заміна, якщо не знайшли, виводять діалогове віконечко. Це не завжди зручно. Змініть, щоб просто виводило повідомлення на QLabel в діалозі пошуку чи заміни.

- Щоб програма була зручна, вона повинна відповідати певним стандартам на поведінку. Один із них — як переключається фокус між віджетами. Ознайомтеся із питанням тут: `http://qt-project.org/doc/qt-5/designer-tab-order.html` та впорядкуйте порядок переключення вашої програми.
- Функції роботи з файлами не аналізують всі можливі помилки! Виправте це.
- В продовження попереднього завдання. Зокрема, збереження файлів слід робити з використанням QSaveFile, (або, що менш зручно для цієї задачі, QTemporaryFile) щоб не псувати дані під час запису: `http://qt-project.org/doc/qt-5/qsavefile.html`, `http://qt-project.org/doc/qt-5/qtemporaryfile.html`.
- Крім пошуку за стрічкою, часто зручним є пошук за *регулярним виразом*. Реалізуйте таку можливість. Детальніше див. документацію на QRegExp: `http://qt-project.org/doc/qt-5/QRegExp.html`. Регулярні вирази Qt моделювалися за Perl, тому також корисним може бути наступне посилання: `http://en.wikipedia.org/wiki/Regexp` і `http://en.wikipedia.org/wiki/PCRE`.

- QTextEdit підтримує "багате" форматування. Додайте можливість зміни стилю тексту: bold, italic, тощо. Додайте можливість зміни шрифтів та кольору тексту. Див. `http://qt-project.org/doc/qt-5/qfontdialog.html` і `http://qt-project.org/doc/qt-5/qcolordialog.html`.
- Кожен символ кінця рядка в QTextEdit розпочинає новий абзац. Додайте можливість незалежного форматування абзаців — ширини, відступів, віддалі між рядками, тощо, як ось в текстових процесорах типу MS Word.
- Важливим елементом текстового редактора є можливість друку. Реалізуйте її. Щоб знати, як це зробити, див. документацію по QPrinter: `http://qt-project.org/doc/qt-5/QPrinter.html` та QPrintPreviewDialog: `http://qt-project.org/doc/qt-5/qprintpreviewdialog.html`.
- Нарешті, реалізуйте ще одну можливість — збереження налаштувань. Поміж них — розташування діалогів, вибрані галочки, тощо, так, щоб вони зберігалися між запусками програми. Для цього зверніться до класів QSettings: `http://qt-project.org/doc/qt-5/qsettings.html` та QCommandLineParser: `http://qt-project.org/doc/qt-5/qcommandlineparser.html`.

Додаткові домашні завдання:

- Реалізуйте локалізацію програми. Перекладіть її українською та російською (або іншою, на вибір, мовою). Вважаємо, що базово програма писалася англійською. Вибрану мову зберігайте в конфігурації! Деталі див:
  `http://qt-project.org/doc/qt-4.8/linguist-hellotr.html` — "Hello tr() Example";
  `http://qt-project.org/doc/qt-5/internationalization.html` — "Internationalization with Qt";
  `http://qt-project.org/doc/qt-5/qtlinguist-index.html` — "Qt Linguist Manual";
  `http://qt-project.org/doc/qt-5/i18n-source-translation.html` — "Writing Source Code for Translation".
- Реалізуйте допомогу до вашої програми. Деталі див. "The Qt Help Framework":
  `http://qt-project.org/doc/qt-4.8/qthelp-framework.html`.
- Реалізуйте можливість заміни іконок програми. Деталі див. "The Qt Resource System":
  `http://qt-project.org/doc/qt-5/resources.html`, зокрема "External Binary Resources".